Aula 04:

Um programa Java segue uma estrutura bem definida, começando com a declaração do pacote, importações necessárias e a definição da classe principal que contém o método `main`.

Java

```java
package com.input.app; // Declaração do pacote onde a classe reside

import java.util.Scanner; // Importa a classe Scanner do pacote java.util

public class App { // Declaração da classe principal
    public static void main(String[] args) throws Exception { // Método princip
        // O código do programa vai aqui
    }
}
```

- **`package com.input.app;`**: Define o pacote ao qual a classe `App` pertence. Pacotes são usados para organizar classes e evitar conflitos de nomes.
- **`import java.util.Scanner;`**: Permite que a classe `Scanner`, que está em outro pacote (`java.util`), seja usada diretamente pelo nome em `App.java`.
- **`public class App { ... }`**: Define a classe principal do programa. Em Java, todo o código reside dentro de classes.
- **`public static void main(String[] args) throws Exception { ... }`**: É o ponto de entrada de qualquer aplicação Java. Quando você executa um programa Java, a Máquina Virtual Java (JVM) procura e executa este método.
  - `public`: O método é acessível de qualquer lugar.
  - `static`: O método pertence à classe, não a uma instância específica dela, e pode ser chamado sem criar um objeto da classe.
  - `void`: O método não retorna nenhum valor.
  - `String[] args`: Permite que argumentos de linha de comando sejam passados para o programa.
  - `throws Exception`: Indica que o método pode lançar exceções (erros) que não são tratadas internamente pelo próprio método. É uma forma de delegar a responsabilidade de tratamento dessas exceções para o código que chamar o método `main`. Em programas simples, é comum usar `throws Exception` para evitar a necessidade de blocos `try-catch` complexos, mas em aplicações maiores, o tratamento de exceções é feito de forma mais granular.

1. Organização de Projetos e Ambiente de Desenvolvimento (VSCode)

O texto menciona o uso de um programa especial, como o VSCode, para organizar projetos. Em Java, o ambiente de desenvolvimento integrado (IDE) é crucial para a produtividade. Embora o VSCode seja uma ferramenta poderosa e versátil, para o desenvolvimento Java, IDEs como **IntelliJ IDEA**, **Eclipse** e **NetBeans** são mais comumente utilizados e oferecem um suporte mais robusto, incluindo:

- **Gerenciamento de Dependências**: Ferramentas como Maven ou Gradle são integradas para gerenciar bibliotecas e dependências do projeto.
- **Refatoração Avançada**: Recursos que permitem reestruturar o código de forma segura.
- **Depuração (Debugging)**: Ferramentas sofisticadas para encontrar e corrigir erros.
- **Integração com Servidores de Aplicação**: Facilita o desenvolvimento de aplicações web.

Para abrir um projeto Java em qualquer uma dessas IDEs, o processo é similar ao descrito: selecionar a pasta raiz do projeto. A estrutura de pastas de um projeto Java geralmente segue convenções, como a estrutura de diretórios do Maven (`src/main/java`, `src/test/java`, etc.), que facilita a organização e a colaboração.

2. Variáveis: Tipos Primitivos e Classes (Objetos)

Em Java, a distinção entre **tipos primitivos** e **tipos de referência (classes/objetos)** é fundamental. O texto descreve bem essa diferença:

Tipos Primitivos

São os blocos de construção básicos da linguagem, armazenando valores diretamente na memória. Em Java, existem oito tipos primitivos, mas vimos esses na aula 03:

Tipo Primitivo	Descrição	Tamanho (bits)	Valor Padrão	Exemplo de Uso
int	Inteiro padrão	32	0	`int populacao = 100000;`
`double`	Ponto flutuante de precisão dupla	64	0.0d	`double salario = 2500.50;`

Tipos de Referência (Classes/Objetos)

São tipos mais complexos que armazenam referências (endereços de memória) para objetos. O texto menciona `String` como um exemplo. Em Java, `String` não é um tipo primitivo, mas sim uma **classe** do pacote `java.lang`. Isso significa que variáveis `String` são objetos e possuem métodos associados para manipulação de texto (ex: `length()`, `toUpperCase()`, `substring()`).

Curiosidade: Objetos `String` em Java são **imutáveis**. Uma vez criados, seu conteúdo não pode ser alterado. Qualquer operação que pareça modificar uma `String` na verdade cria uma nova `String` com o conteúdo modificado.

3. Números Decimais: O Tipo `double`

O texto destaca o uso do `double` para números com casas decimais, chamados de **pontos flutuantes**. Em Java, `double` é o tipo de ponto flutuante de precisão dupla, oferecendo maior precisão e um alcance maior de valores em comparação com `float`.

Curiosidade: Embora `double` seja excelente para cálculos científicos e de engenharia, ele não é recomendado para cálculos financeiros que exigem precisão exata. Isso ocorre porque os números de ponto flutuante podem ter pequenas imprecisões devido à sua representação binária. Para cálculos financeiros, a classe `java.math.BigDecimal` é a escolha preferida, pois representa números decimais com precisão arbitrária.

4. Comentários no Código

Os comentários são essenciais para a legibilidade e manutenção do código. Em Java, existem três tipos de comentários:

- **Comentário de linha única**: Começa com `//` e vai até o final da linha.
- **Comentário de múltiplas linhas**: Começa com `/*` e termina com `*/`.
- **Comentário de documentação (Javadoc)**: Começa com `/**` e termina com `*/`. É usado para gerar documentação HTML automaticamente a partir do código-fonte.

Curiosidade: O Javadoc é uma ferramenta poderosa que permite aos desenvolvedores criar documentação abrangente para suas APIs. É uma prática recomendada para projetos Java bem estruturados.

5. Instruções, Concatenação e Separador Decimal

Concatenação de Strings

Em Java, o operador `+` é usado para concatenar (juntar) strings. Se um dos operandos for uma `String`, o outro operando (mesmo que seja um número ou um tipo primitivo) será automaticamente convertido para `String` antes da concatenação.

Separador Decimal

O texto corretamente aponta que, ao digitar números decimais no código Java, usamos o **ponto final (.)** como separador decimal, não a vírgula.

6. Interação com o Usuário: Classe `Scanner`

Para receber entrada do usuário via console, a classe `Scanner` do pacote `java.util` é a ferramenta padrão em Java.

Java

```java
// Exemplo baseado no código fornecido na imagem
package com.input.app;

import java.util.Scanner;

public class App {
    public static void main(String[] args) throws Exception {
        // Declaração de variáveis
        String nome;
        int idade;
        double altura;
        String email;

        // Entrada de dados
        Scanner sc = new Scanner(System.in); // Cria um objeto Scanner para rec

        System.out.println("Informe o seu nome:");
        nome = sc.nextLine(); // nextLine() é um método que recebe uma string

        System.out.println("Informe a sua idade:");
        idade = sc.nextInt(); // nextInt() é um método que recebe um número int

        System.out.println("Informe sua altura em metros:");
        altura = sc.nextDouble(); // nextDouble() é um método que recebe um núm

        // Limpeza de buffer: Consumir a quebra de linha pendente após nextInt(
        sc.nextLine(); // Importante para evitar problemas com o próximo nextLi

        System.out.println("Informe o seu email:");
        email = sc.nextLine(); // nextLine() é um método que recebe uma string
```

```
        // Exibição dos dados coletados
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade);
        System.out.println("Altura: " + altura);
        System.out.println("Email: " + email);

        sc.close(); // Fecha o Scanner para liberar recursos
    }
}
```

Limpeza de Buffer

O texto menciona a necessidade de uma "limpeza" antes de pedir uma nova informação, como um e-mail, após ler um número. Isso é uma **observação crucial** ao usar `Scanner`. Métodos como `nextInt()`, `nextDouble()`, `next()` leem apenas o token esperado (o número ou a palavra), mas deixam o caractere de quebra de linha (`\n`) no buffer de entrada. Se um `nextLine()` for chamado em seguida, ele lerá essa quebra de linha vazia e não esperará por uma nova entrada do usuário. A solução é adicionar um `sc.nextLine()` extra para consumir essa quebra de linha pendente, como mostrado no exemplo acima.

Erro Comum: `InputMismatchException` com Números Decimais

Um erro frequente ao usar `Scanner` para ler números decimais (`double` ou `float`) é a `InputMismatchException`. Embora no código Java o separador decimal seja o ponto (`.`), ao executar o programa no terminal, a entrada do usuário pode exigir a **vírgula (`,`)** como separador decimal, dependendo da configuração de localidade (locale) do sistema operacional. Se o usuário digitar `10.5` em um sistema configurado para usar vírgula, ou `10,5` em um sistema configurado para usar ponto, o `Scanner` não conseguirá interpretar a entrada e lançará essa exceção.

Para contornar isso, pode-se configurar o `Scanner` para usar uma localidade específica (por exemplo, `Locale.US` para ponto ou `Locale.getDefault()` para a localidade do sistema) ou instruir o usuário sobre o formato esperado.